Software Requirements Specification

for

LifeSamadhan

(Unified Local Service Provider Management System)

Version 1.0

Prepared by Project Team 25

Name	PRN
Suraj Rawat	250840320210
Ishan Raizada	250840320081
Dipak Chandrakant Firake	250840320059
Pratik Kailas Barse	250840320138
Shashibhushan Avdhesh Mishra	250840320184

Centre for Development of Advanced Computing

Raintree Marg, Near Bharati Vidyapeeth, Opp. Kharghar Railway Station, Sector 7,
CBD Belapur, Navi Mumbai - 400 614 - Maharashtra (India)
Phone: +91-22-27565303 Fax: +91-22-2756-0004

Table of Contents

Table of Contents **2**
Revision History **3**
1. Introduction **4**
1.1 Purpose 4
1.2 Intended Audience and Reading Suggestions 4
1.3 Project Scope 4
1.4 References 5
2. Overall Description **5**
2.1 Product Perspective 5
2.2 Product Functions 5
2.3 User Classes and Characteristics 6
2.4 Operating Environment 6
2.5 Design and Implementation Constraints 7
2.6 User Documentation 7
2.7 Assumptions and Dependencies 7
3. System Features **8**
3.1 User Registration & Login 8
3.2 Admin Approval Management 8
3.3 Customer Service Request 8
3.4 Auto-Assignment Engine 9
3.5 Provider Acceptance Window 9
3.6 OTP Verification 9
3.7 Service Completion 9
3.8 Payment Processing 9
3.9 Rating & Feedback 9
4. External Interface Requirements **10**
4.1 User Interfaces 10
4.2 Hardware Interfaces 10
4.3 Software Interfaces 10
4.4 Communications Interfaces 10
5. Other Nonfunctional Requirements **11**
5.1 Performance Requirements 11
5.2 Safety Requirements 11
5.3 Security Requirements 11
5.4 Software Quality Attributes 11
Appendix A: Glossary **13**
Appendix B: Analysis Models **14**

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the LifeSamadhan – Unified Local Service Provider Management System and provides detailed description of the system's functionalities. The SRS document also outlines the performance and security requirements for the system.

1.2 Intended Audience and Reading Suggestions

This document is intended for:

• Developers – to understand functional workflow and module behavior

• Testers – to derive test cases and validate requirements

• Project Guide / Evaluators – to assess completeness of requirements

• Administrators – to understand approval workflows and governance model

Readers should begin with Overall Description, then System Features, followed by

Non-Functional Requirements and Appendix models.

1.3 Project Scope

LifeSamadhan is a managed service platform that connects customers with verified

local service providers through a secure and automated workflow. The system focuses on:

• Standardized pricing and transparency

• Provider skill verification and admin approval

• Automated provider assignment instead of manual selection

• OTP validation for service start confirmation

• Payment processing after service completion

• Rating and feedback for quality improvement

The platform digitally empowers skilled and daily wage workers while ensuring trust, safety, and fair opportunity distribution.

1.4 References

- https://spring.io/projects/spring-boot
- https://learn.microsoft.com/aspnet/core/web-api/?view=aspnetcore-7.0
- https://reactjs.org/docs/getting-started.html
- https://dev.mysql.com/doc/
- https://datatracker.ietf.org/doc/html/rfc7519
- https://docs.oracle.com/javase/8/docs/api/java/lang/Object.html
- https://aws.amazon.com/architecture/well-architected/

2. Overall Description

2.1 Product Perspective

LifeSamadhan is a standalone web-based system that follows a managed service model

Customers do not choose providers ,instead the system automatically assigns the best-matched approved provider based on:

• Skill match

• Location mapping

• Availability

• Rating priority

2.2 Product Functions

The system provides following primary functions:

• User Registration & Role-based Login

• Admin approval of provider skills & locations

• Service catalog & price configuration

• Customer service request creation

• Auto assignment of provider

• Service Provider acceptance window within a specified time limit

• OTP verification at customer location

• Service execution tracking

• Payment processing (cash / online simulated)

• Rating & feedback submission

• Notification alerts & reporting

2.3 User Classes and Characteristics

1. Customer

Requests services, makes payment, and submits rating.

2. Service Provider

Registers, submit skills & service locations for admin approval, accepts assigned jobs, performs service, and updates completion status.

3. Administrator

Controls approvals, pricing, locations, provider verification, monitoring and moderation.

2.4 Operating Environment

The operating environment for the project consists of the following hardware and software components:

Hardware:

- A machine with at least 8GB of RAM and a fast processor, such as Intel Core i5 or higher, to ensure smooth and efficient execution of the project.

Software:

- ReactJS for the frontend development.
- Spring Boot and .NET for the backend development
- MySQL for the database management.
- JWT Authentication for security.
- AWS for deployment.

Other Applications:

- Code editor (such as Visual Studio Code, Eclipse)
- Git version control software
- Command line interface (CLI) or terminal
- A browser for testing the application.
- Swagger & Thunder Client for testing APIs
- MySQL Workbench or another database management tool
- An AWS account for deployment.

2.5 Design and Implementation Constraints

• System must support integration of dual backend technologies

• OTP verification is mandatory before service starts

• Service provider cannot be selected manually by customer

• Auto-assignment algorithm must follow fair-distribution model

• Only approved skills & locations are eligible for assignment

2.6 User Documentation

User documentation mainly comprises of Help menu of application. It will give all the minute details about the project, if any user has any query about any module or functionality, one can refer it and see how to operate the application. This report is the complete documentation of our project. It gives complete details about the project, its functionality, users, software used, hardware requirement, environment and so on.

2.7 Assumptions and Dependencies

Assumptions:

•Internet connectivity is available for all users.
• Service providers submit valid identity documents and accurate skill details during registration.
• Customer service locations exist within the approved service area coverage.
• Service providers have mobile devices capable of receiving notifications and OTP messages.
• Payment methods (cash/online) are accessible to customers at service completion.
• Admin approval process is completed within reasonable timeframes to avoid registration delays.
• Service providers maintain their availability status accurately in the system.
• Customers provide accurate contact information and service addresses.

Dependencies:

- A functional email service is required for notifications to be sent.
- Third-party payment processing service for handling online transactions (if implementing actual payments).
- An active internet connection is necessary to access the application and receive notifications.
- AWS deployment infrastructure.
- ReactJS, Spring Boot, .NET and MySQL components are required to run the application.

3. System Features

3.1 User Registration & Login

Description: Allows Customer & Service Provider registration and login.

Priority: High

Functional Requirements:

FR-01 User shall register with name, email, mobile, password and role.

FR-02 Password must be stored using hashing.

FR-03 Service provider profile shall store skills & documents in pending state.

FR-04 System shall generate JWT token on successful login.

3.2 Admin Approval Management

Description: Admin approves service provider skills & service locations.

Priority: High

FR-05 Admin shall approve or reject provider skills.

FR-06 Only approved skills become eligible for assignment.

FR-07 Admin shall configure services, categories, locations and prices.

3.3 Customer Service Request

Description: Customer submits service request.

Priority: High

FR-08 Request shall be stored with status REQUESTED.

FR-09 Auto-assignment process shall be triggered on request creation.

3.4 Auto-Assignment Engine

Description: Provider automatically assigned based on rules.

Priority: Very High

FR-10 Provider must match approved skill & location.

FR-11 Provider must have status AVAILABLE.

FR-12 Higher rated provider receives priority.

FR-13 Assignment status becomes ASSIGNED.

3.5 Provider Acceptance Window

FR-14 Provider must accept request within time limit.

FR-15 On timeout assignment returns to pool.

3.6 OTP Verification

FR-16 OTP must be validated at customer location.

FR-17 On OTP success → status IN_PROGRESS.

3.7 Service Completion

FR-18 Provider marks service as COMPLETED.

3.8 Payment Processing

FR-19 System creates payment entry.

FR-20 Customer confirms payment.

FR-21 Status changes to SUCCESS.

3.9 Rating & Feedback

FR-22 Customer submits rating and feedback.

FR-23 Rating influences future auto-assignment priority.

4. External Interface Requirements

4.1 User Interfaces

The system should provide a very intuitive and simple interface for the user and the administrator.

4.2 Hardware Interfaces

Server side

- The web application will be hosted on a web server which is listening on the web standard port.

Client side

- Monitor screen – the software shall display information to the user via the monitor
- screen
- Mouse – the software shall interact with the movement of the mouse and the mouse
- buttons. The mouse shall activate areas for data input, command buttons and select
- options from menus.
- Keyboard – the software shall interact with the keystrokes of the keyboard. The
- keyboard will input data into the active area of the database.

4.3 Software Interfaces

Server side

An Apache web server will accept all requests from the client and forward it accordingly. A database will be hosted centrally using MySQL.

Client side

An OS which is capable of running a modern web browser which supports JavaScript and HTML5

4.4 Communications Interfaces

The HTPP or HTTPS protocol(s) will be used to facilitate communication between the client and server.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

• System should serve concurrent customer requests smoothly.

• Auto-assignment should execute within acceptable response time.

5.2 Safety Requirements

The system should be reliable and have minimal downtime to ensure that customers can submit service requests and receive notification like service requests accepted and detail of service provider, in a timely manner. The user interface should be intuitive and user-friendly to prevent errors or misunderstandings while using the app. The app should have robust errors handling mechanisms to deal with unexpected situations and prevent any harm to the user or loss of data. Data integrity is maintained across both backends .

5.3 Security Requirements

The project would include measures to protect confidentiality, integrity, and availability of sensitive information such as personal information. Some of the security requirements are:

- Authentication: A secure authentication system that ensures only authorized users can access the system.

- Authorization: An authorization mechanism to determine what actions a user can perform within the system based on their role and permissions.

- Encryption: Data in transit and at rest should be encrypted to protect sensitive information from being intercepted or accessed by unauthorized parties.

- Access Control: A mechanism to control access to sensitive information within the system, including the ability to set permissions for different users and roles.

5.4 Software Quality Attributes

The software quality attributes for the project include the following:

- Usability: The user interface should be intuitive, easy to navigate and user-friendly, allowing users to easily submit service requests and review it.

- Reliability: The system should be reliable and provide accurate results, ensuring that service requests are submitted and reviewed accurately and efficiently.

- Performance: The system should perform efficiently and respond quickly to user requests, allowing for fast and seamless service request submissions and reviews.

- Scalability: The system should be scalable, allowing for an increasing number of users ,services and service location as the demand grows.

- Maintainability: The system should be maintainable, with easy-to-update code and well-documented processes, ensuring that updates and improvements can be made quickly and efficiently.

- Security: The system should have strong security features, protecting sensitive information and ensuring the privacy and confidentiality of user data.

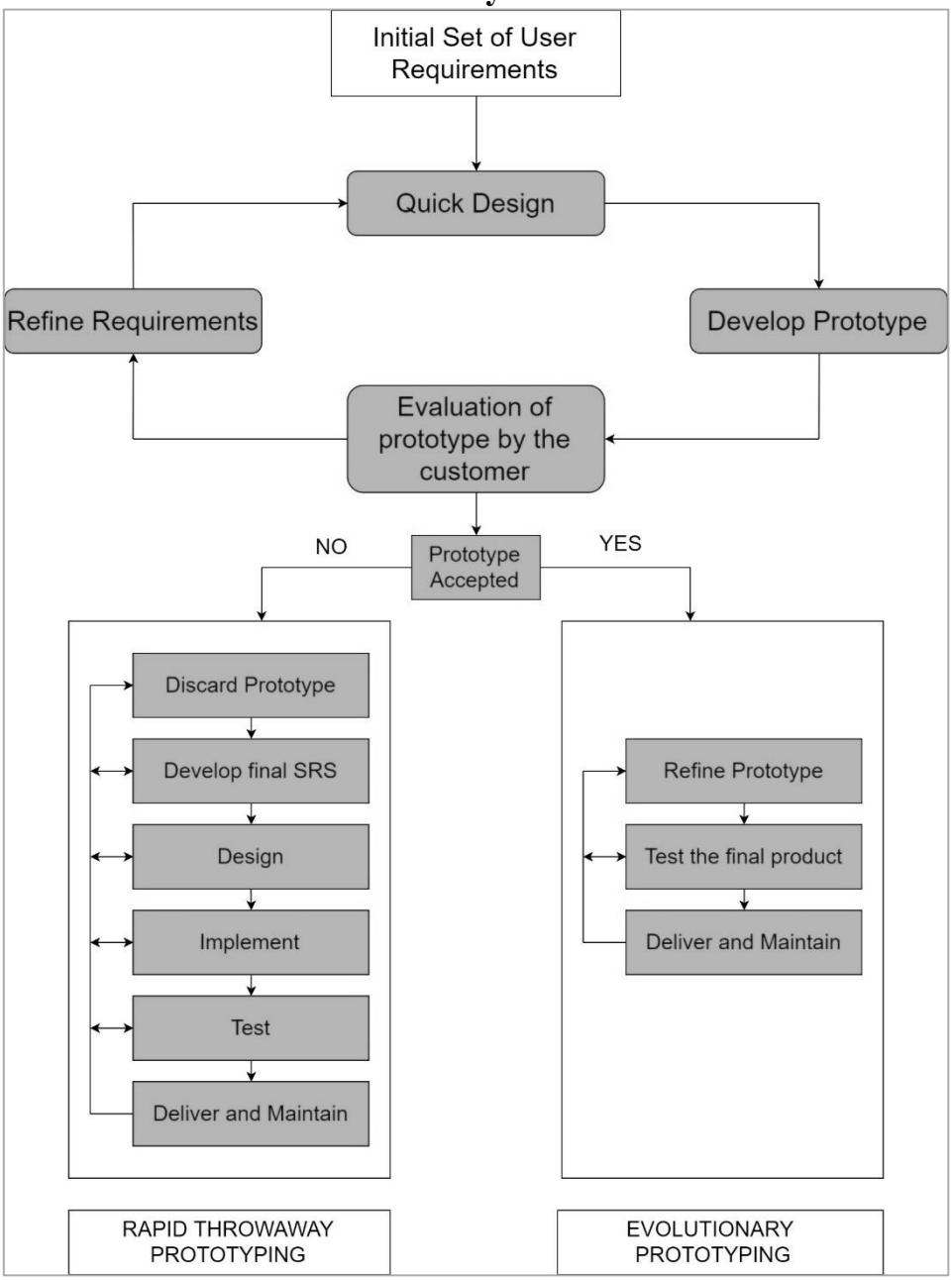
- Compliance: The system should comply with relevant regulations and standards, ensuring that it operates within the bounds of the law and industry best practices.

Appendix A: Glossary

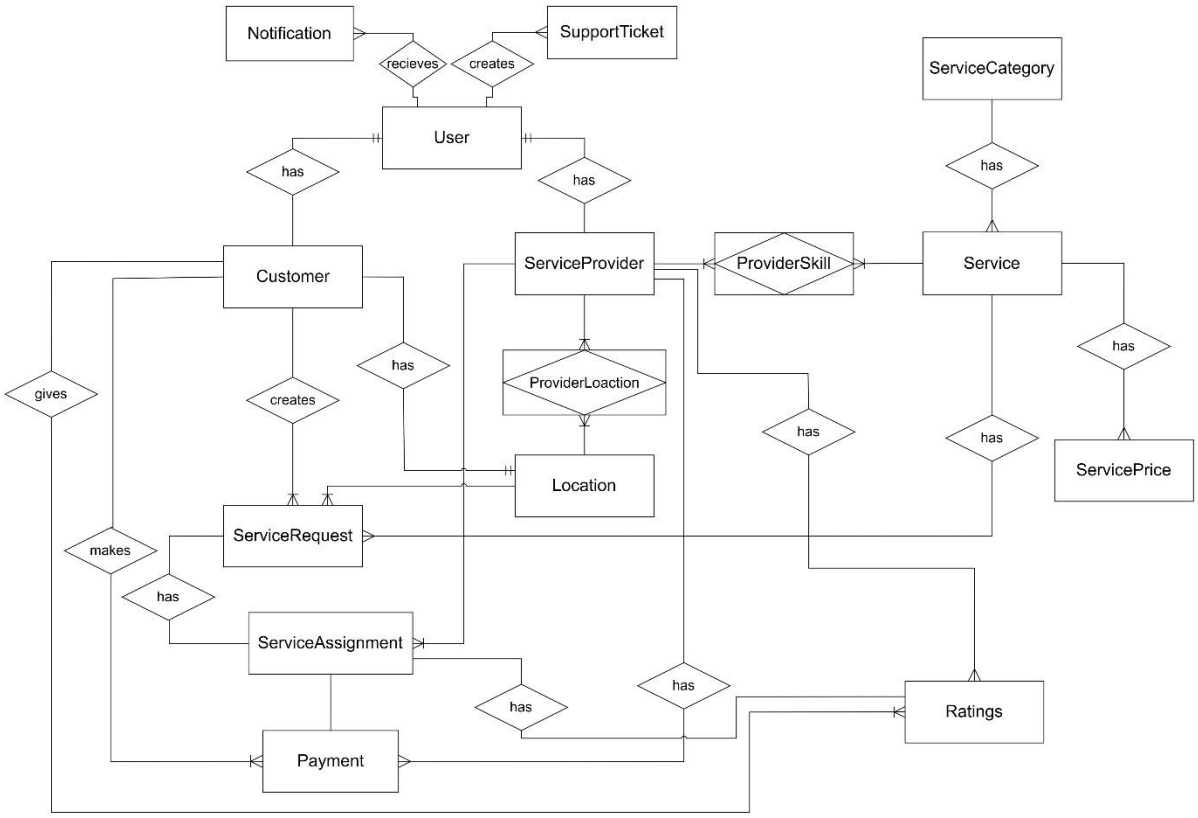
Sr. No.	Abbreviation	Full Form
1.	API	Application Programming Interface
2.	AWS	Amazon Web Service
3.	CLI	Command line Interface
4.	GB	Gigabyte
5.	HTML	Hypertext Markup Language
6.	HTTP / HTTPS	Hypertext Transfer Protocol / Hypertext Transfer Protocol Secure
7.	ID	Identification
8.	JS	JavaScript
9.	JWT	JSON Web Token
10.	OS	Operating System
11.	RAM	Random Access Memory
12.	SQL	Structured Query Language
13.	SRS	Software Requirement Specification
14.	URL	Uniform Resource Locator
15.	ER	Entity Relationship

Appendix B: Analysis Models

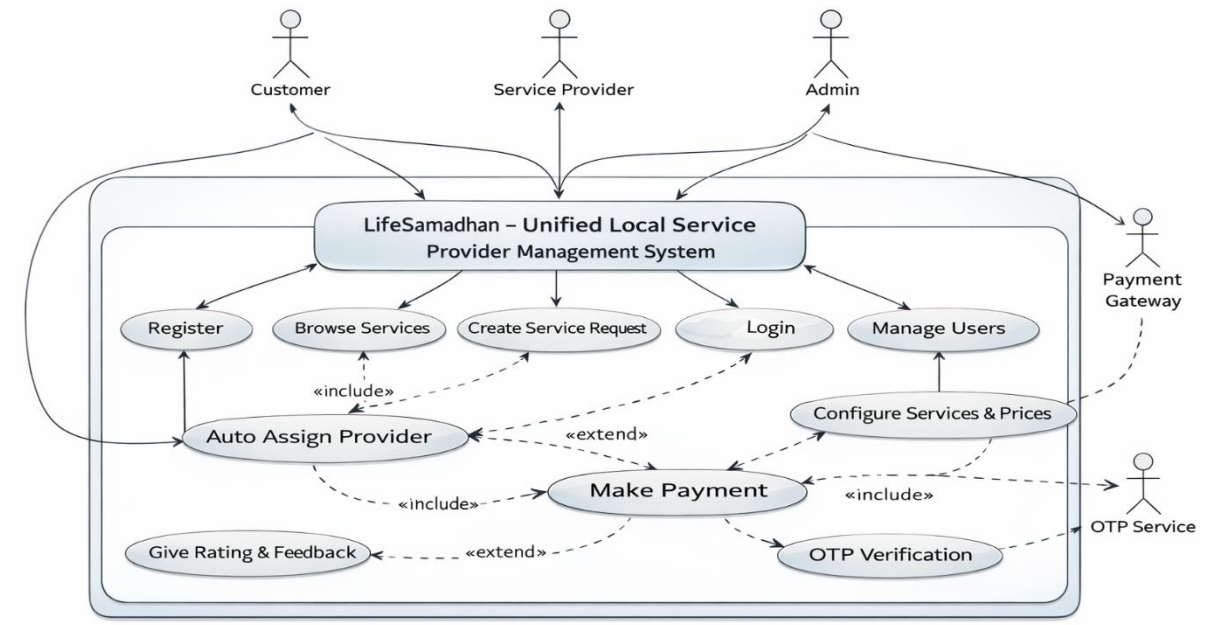
- **Software Development Approach in Our System**



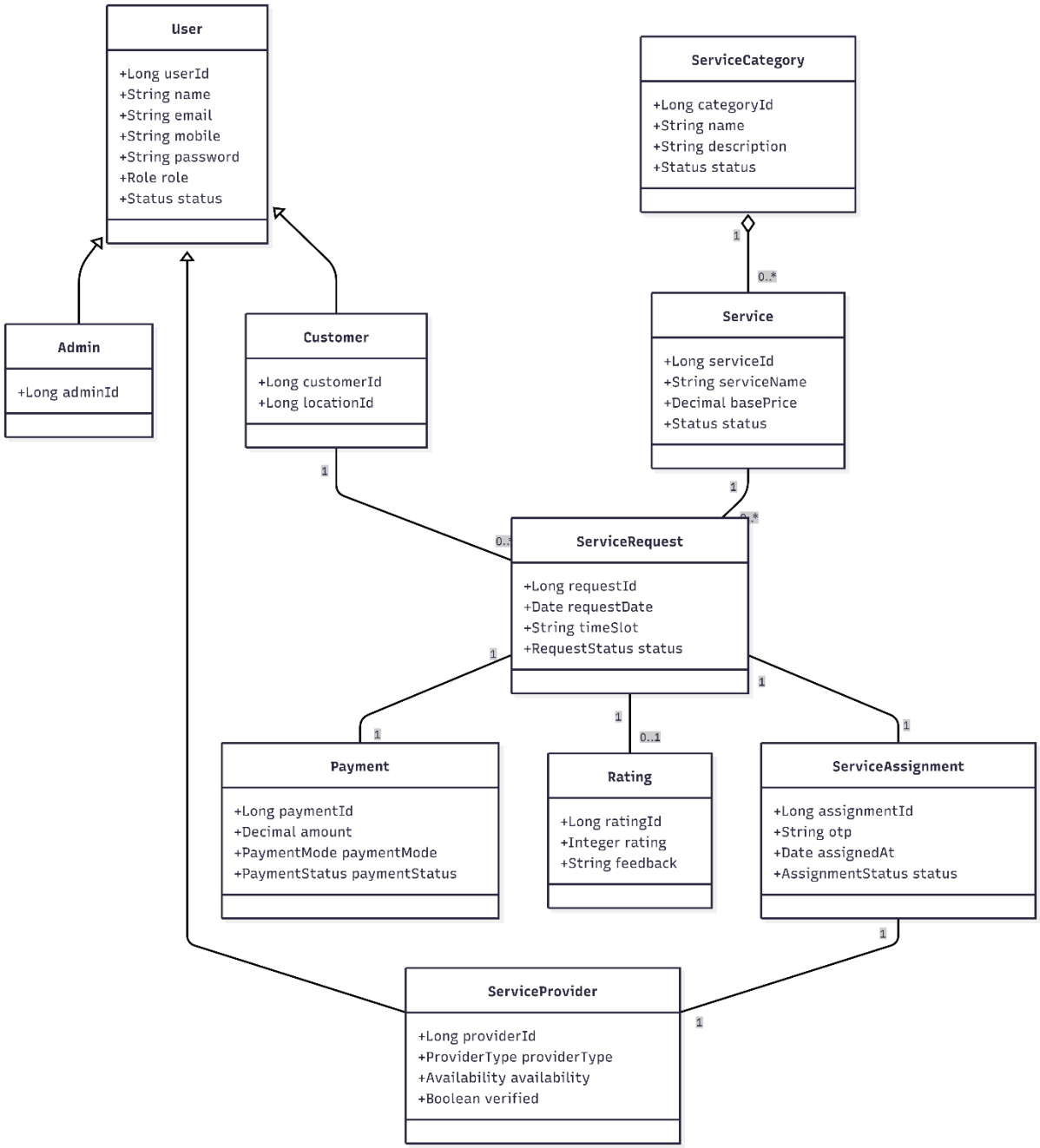
ER Diagram



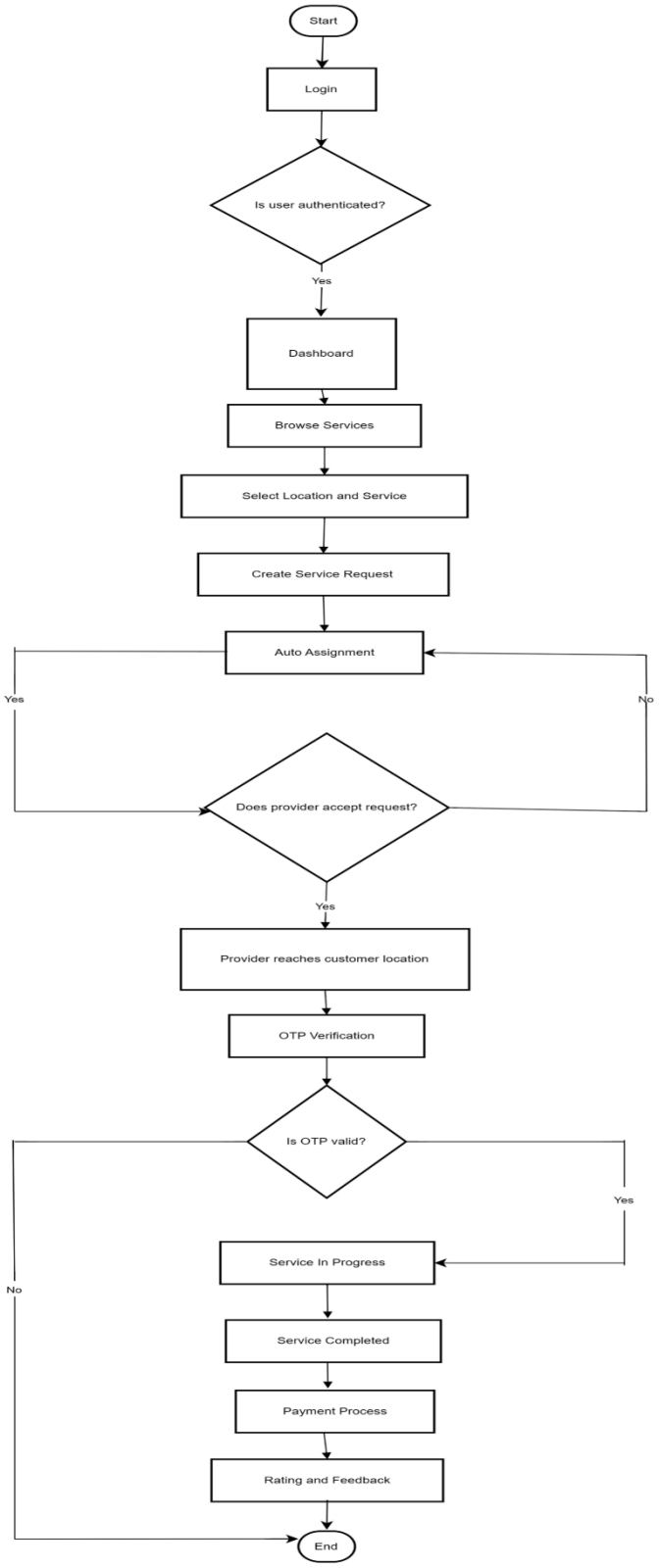
Use-Case Diagram



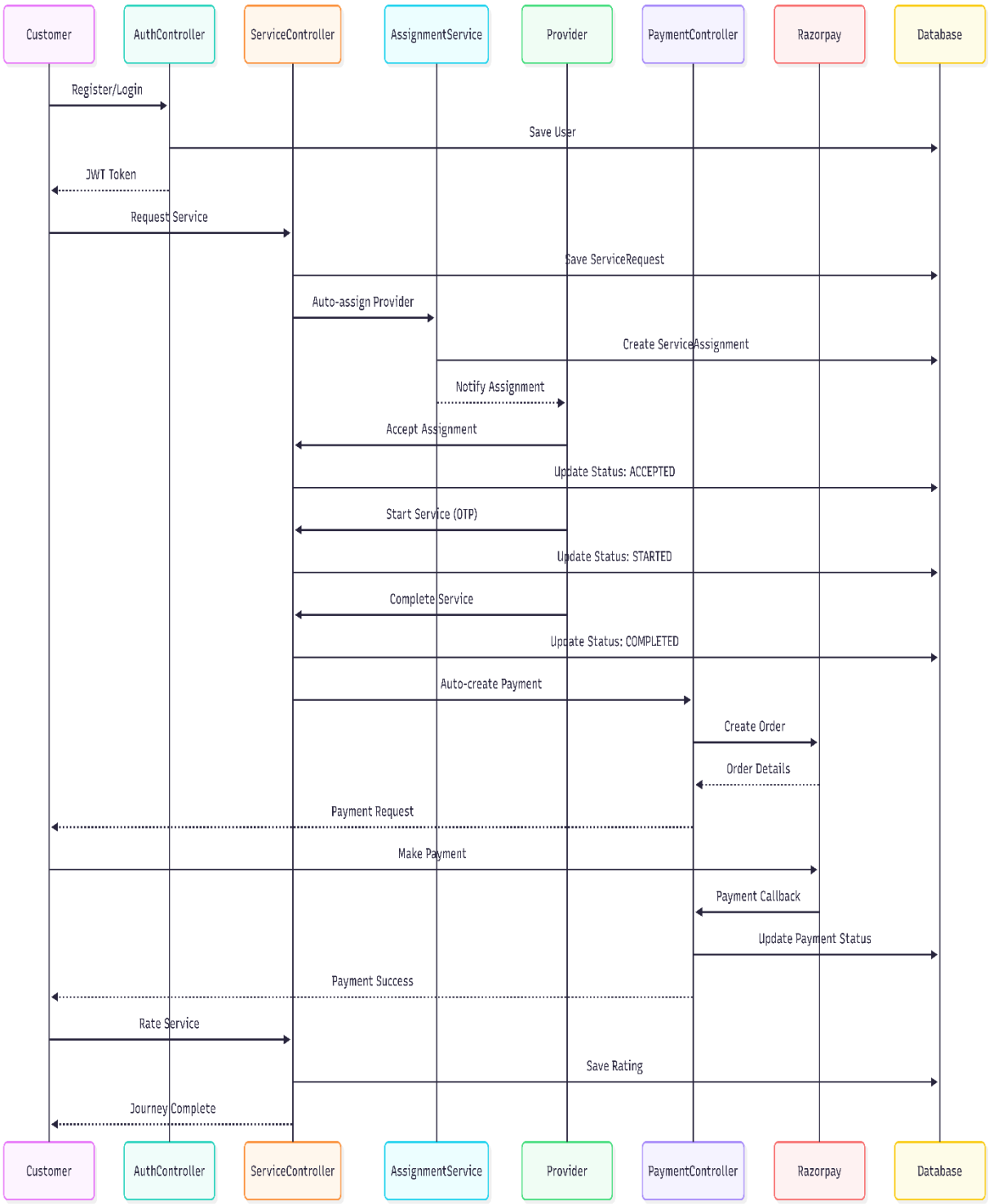
Class Diagram



Activity Diagram

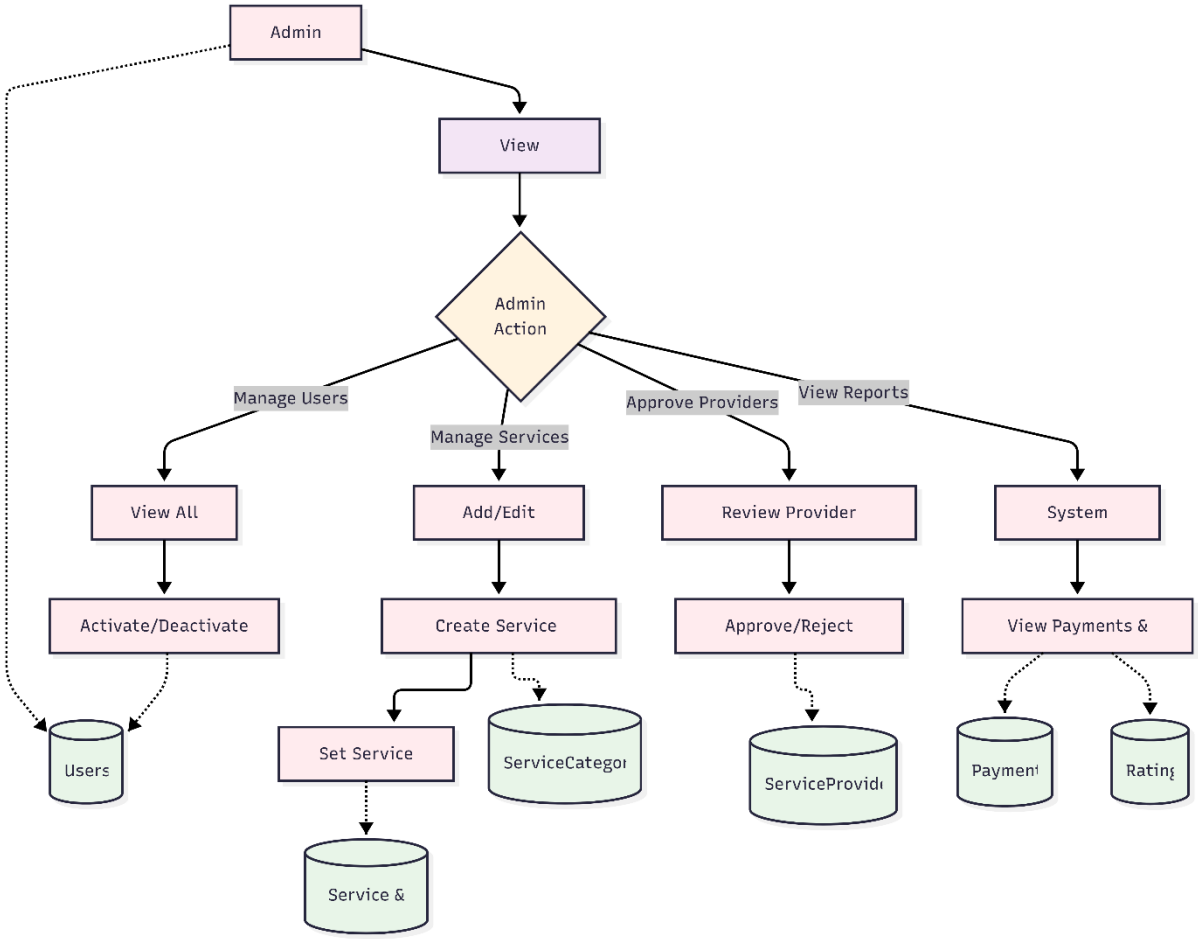


Sequence Diagram

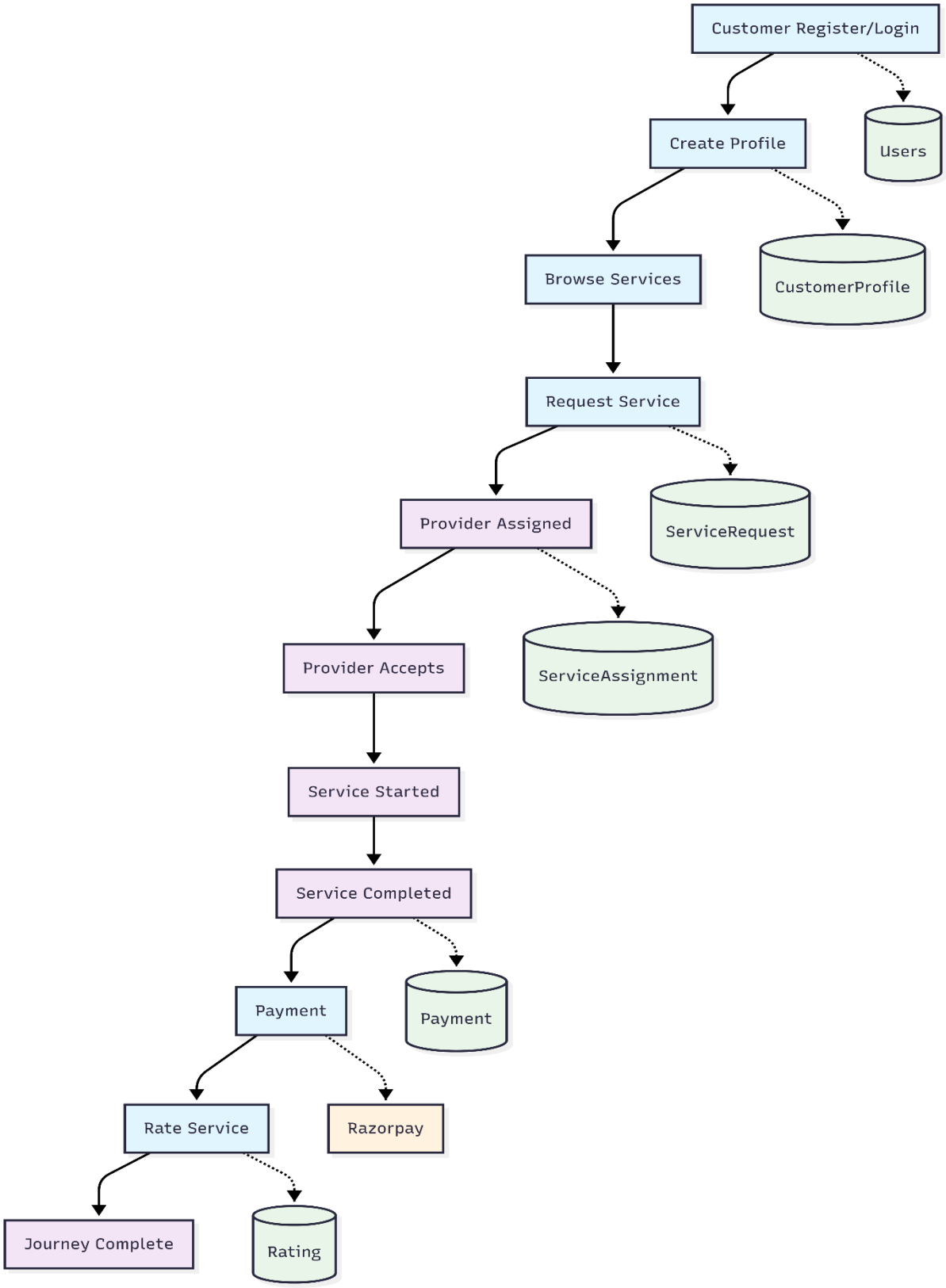
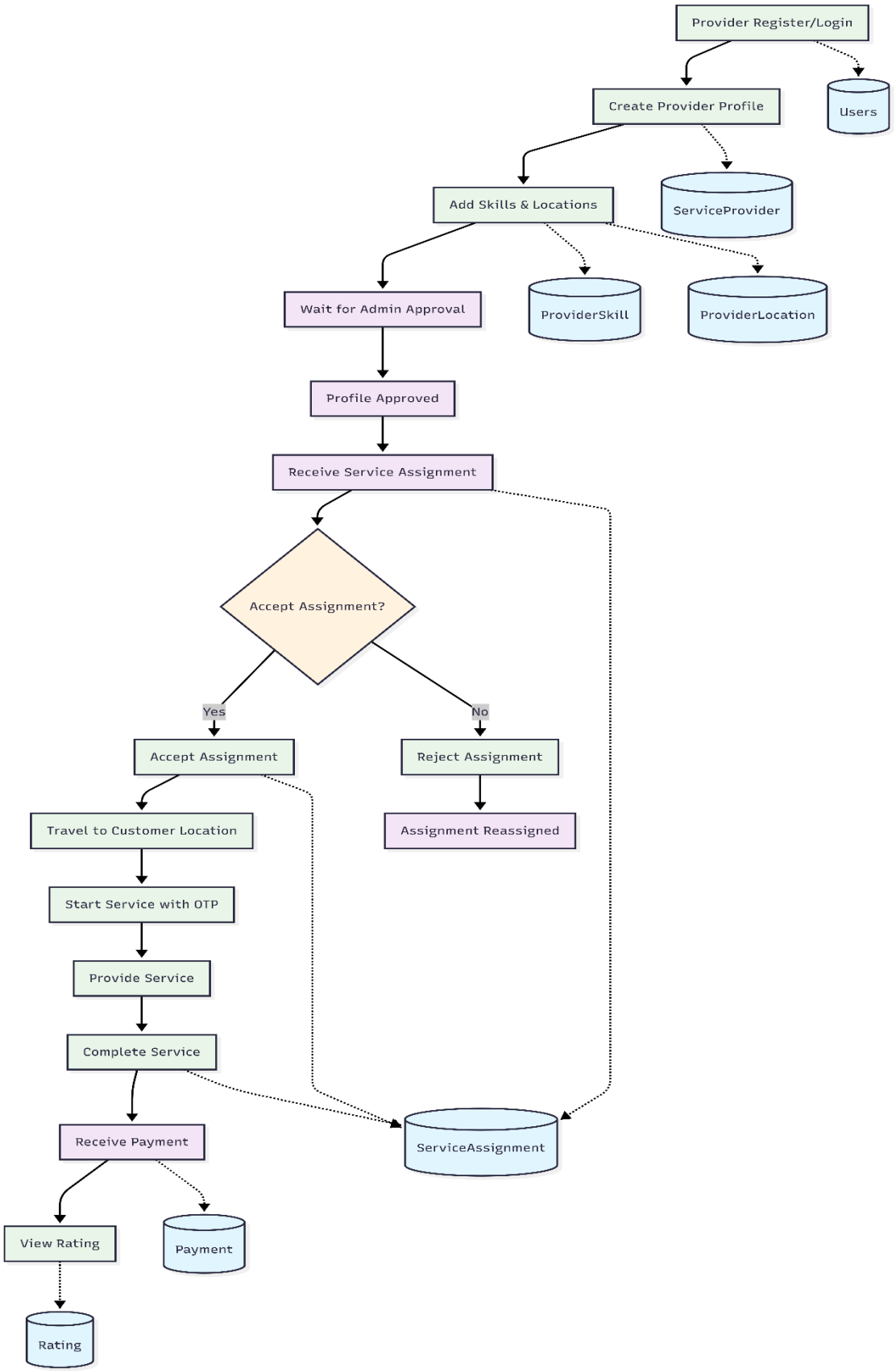


Data Flow Diagram (DFD)

1)ADMIN



2)CUSTOMER


Customer Register/Login
Users
Create Profile
CustomerProfile
Browse Services
Request Service
ServiceRequest
Provider Assigned
ServiceAssignment
Provider Accepts
Service Started
Service Completed
Payment
Payment
Rate Service
Razorpay
Journey Complete
Rating

3)SERVICE PROVIDER